

"The agent reasons, MCP connects, the microservice stays the authority."

MODULE 11

Model Context Protocol (MCP) and Microservices

Standardized AI access to enterprise capabilities.

MCP concepts

Clients and
servers

Tools,
resources,
prompts

Calling Spring
Boot services

Designing
MCP-enabled
services

Agent-to-
backend
integration

Security and
authorization

Enterprise MCP
patterns

LEARNING OBJECTIVES

MCP architecture, servers, clients, tools, microservices, security and design patterns.



BY THE END OF THIS MODULE, YOU CAN

1. Explain why MCP exists
2. Describe MCP host / client / server roles
3. Differentiate tools, resources and prompts
4. Map REST APIs to MCP tools
5. Design useful MCP tool contracts
6. Connect MCP to Spring Boot microservices
7. Design scalable MCP servers
8. Apply authentication and authorization
9. Integrate agents with multiple services
10. Handle long-running operations
11. Monitor MCP interactions
12. Identify practical enterprise patterns

REMEMBER The agent reasons, MCP connects, the microservice stays the authority.



FIVE SECTIONS

1 MCP fundamentals
Host → Client → Server

2 Capabilities
Tools · Resources · Prompts

3 Microservices integration
MCP → REST API → Spring Boot

4 Production architecture
Gateway · Security · Scaling · Observability

5 Agentic orchestration
Agent → MCP → multiple microservices → business outcome

KEY TAKEAWAY MCP gives agents a standard, secure way to reach microservice capabilities.

FROM MODULE 10 TO MODULE 11

Agents need tools — MCP gives them one standard, governed way to reach ours.

MODULE 10 GAVE US

- ✓ Agents that plan and call tools
- ✓ Narrow, typed tool definitions
- ✓ Bounded loops and stop conditions
- ✓ Safe tool endpoints in Spring Boot
- ✓ Policy, approval and audit

NOW Standardize how agents discover and call microservice capabilities.



OUR PATH THROUGH MODULE 11

- 1 What MCP is and how a request flows
- 2 Tools, resources, prompts, discovery
- 3 Wrap Spring Boot APIs as MCP tools
- 4 Design good tool contracts
- 5 Scale, gateway and security
- 6 Reliability and tracing
- 7 Orchestrate services through MCP

Part 1

Part 2

Part 3

Part 3

Part 4

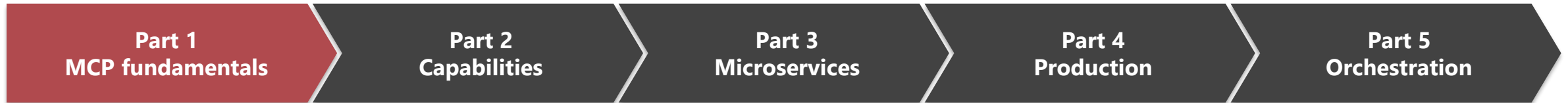
Part 4

Part 5

KEY TAKEAWAY Module 10 designed agents; Module 11 connects them to microservices through one standard protocol.

PART 1: MCP FUNDAMENTALS

Why a protocol — and which roles it defines.



SO FAR

- ✓ Agents call tools to reach services
- ✓ Tools must be narrow and typed
- ✓ Services keep business authority

NEXT QUESTION

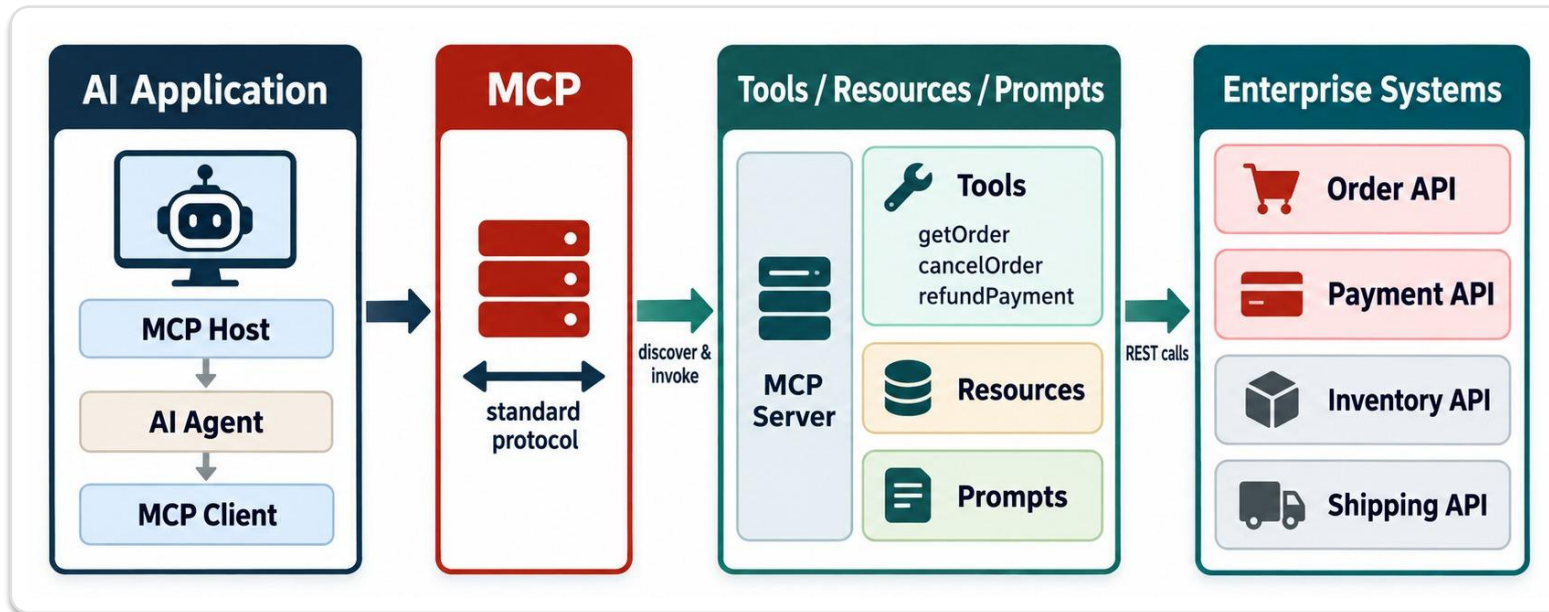
Why do agents need a standard protocol to reach tools and context?

IN THIS PART

- 1 What MCP is
- 2 Why it matters for agentic AI
- 3 Host, client and server
- 4 How a request flows
- 5 The protocol messages — in JSON

WHAT IS MODEL CONTEXT PROTOCOL?

Why MCP exists and what problem it solves.



An open standard

How AI applications discover and use tools, resources and prompts.

Not a replacement

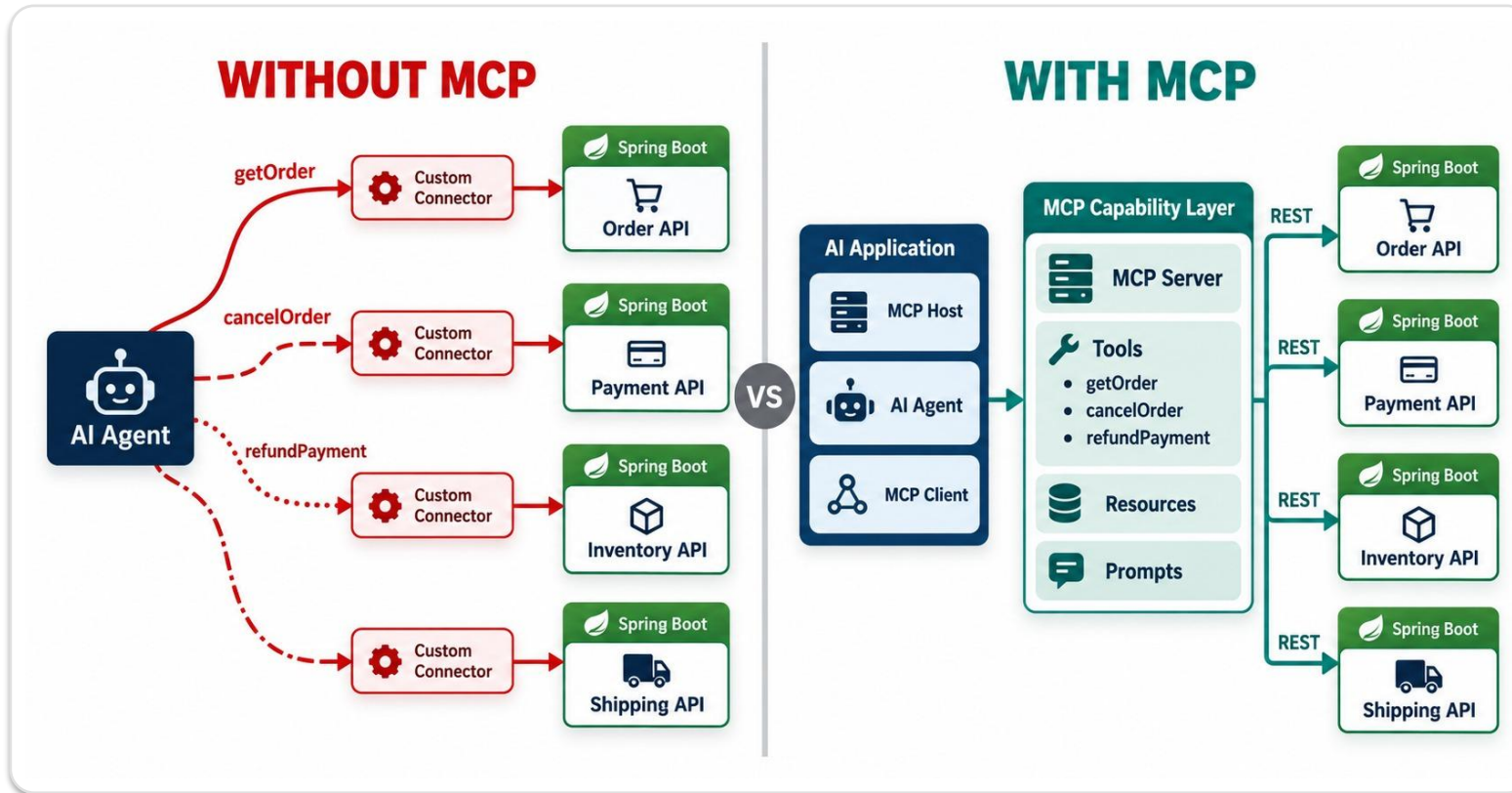
Your REST APIs, security and business logic stay where they are.

ANALOGY Like a common plug shape for AI integrations.

KEY TAKEAWAY MCP is a standard protocol connecting AI applications to capabilities and context.

WHY MCP MATTERS FOR AGENTIC AI

Standardized tool and context access instead of custom integrations per agent.



Without MCP

$N \text{ agents} \times M \text{ systems} = N \times M$ custom connectors to maintain.

With MCP

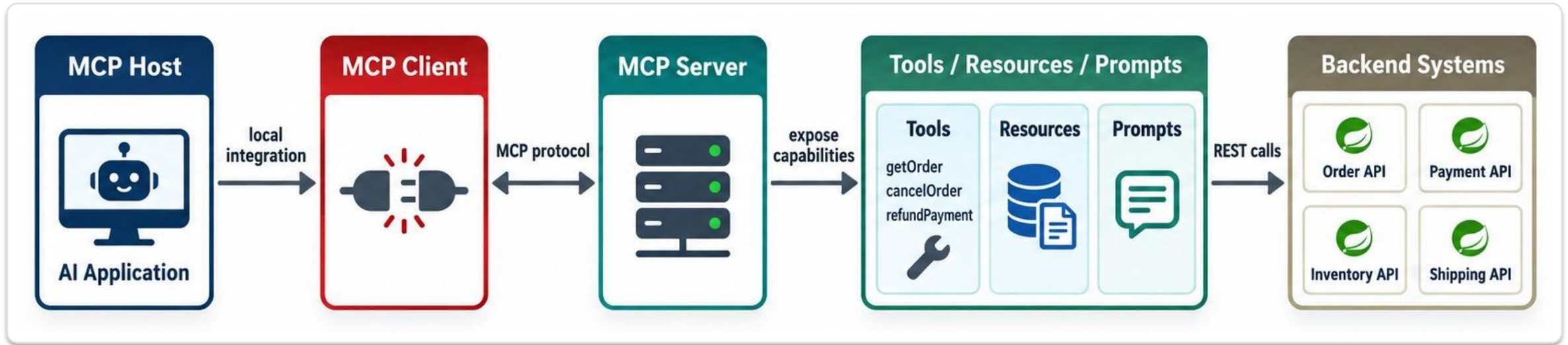
Each system exposes one MCP server; any client can use it.

MOST USEFUL when many AI clients need the same capabilities.

KEY TAKEAWAY MCP replaces one-off connectors with one standard way to find and call capabilities.

MCP ARCHITECTURE: HOST, CLIENT, AND SERVER

The core architectural roles.



Host

The AI application — Copilot, an IDE or your agent.

Client

Inside the host; one connection per MCP server.

Server

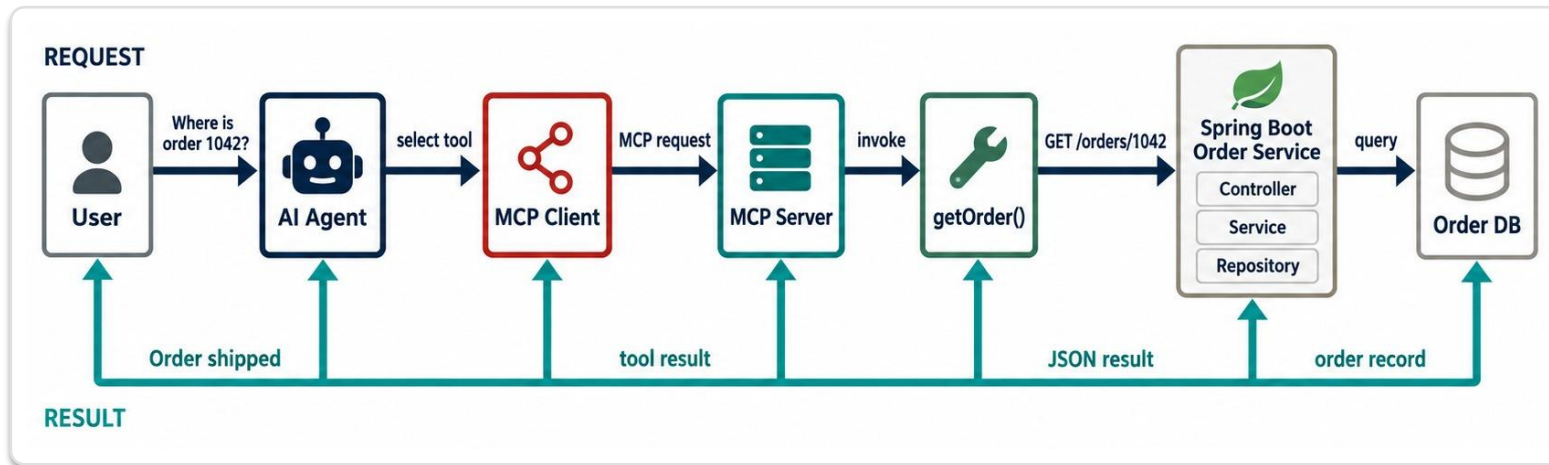
Exposes capabilities and calls the real backend.

IMPORTANT The server holds no business data of its own.

KEY TAKEAWAY Host holds the AI experience, the client speaks MCP, the server exposes capabilities.

HOW AN MCP REQUEST FLOWS

AI host → MCP client → MCP server → backend capability → result.



STEP BY STEP

- ✓ Agent decides to call get_order
- ✓ Client sends tools/call
- ✓ Server calls GET /api/v1/orders/{id}
- ✓ Service validates, reads, returns
- ✓ Result goes back to the agent

SEPARATION Reasoning, protocol and business logic stay apart.

KEY TAKEAWAY A tool call flows agent → MCP → REST → Spring Boot, and the result returns to the agent.

WHAT GOES OVER THE WIRE

MCP messages are JSON-RPC: discover the tools, then call one.

tools/list — the result

```
{
  "jsonrpc": "2.0", "id": 1,
  "result": {
    "tools": [{
      "name": "get_order",
      "description": "Status and items of one order. Read-only.",
      "inputSchema": {
        "type": "object",
        "properties": { "orderId": { "type": "string" } },
        "required": ["orderId"]
      },
      "annotations": { "readOnlyHint": true }
    }]
  }
}
```

Discovery

tools/list returns names, descriptions and schemas.

✓ tools/call — request and result

```
{ "jsonrpc": "2.0", "id": 7,
  "method": "tools/call",
  "params": { "name": "get_order",
              "arguments": { "orderId": "ORD-1045" } } }

{ "jsonrpc": "2.0", "id": 7,
  "result": {
    "structuredContent": {
      "orderId": "ORD-1045", "status": "SHIPPED" },
    "content": [{ "type": "text",
                  "text": "Order ORD-1045 is SHIPPED" }],
    "isError": false } }
```

Call

tools/call sends the tool name and validated arguments.

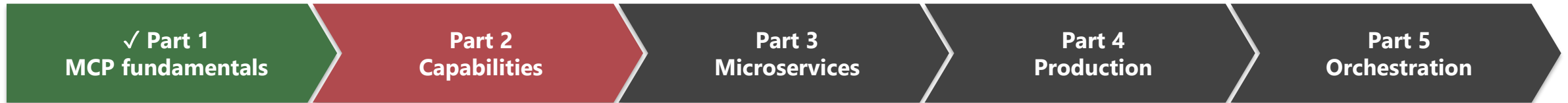
Result

Structured data for code, text for the model — or isError.

KEY TAKEAWAY MCP is plain JSON-RPC: list capabilities, call a tool, get structured results.

PART 2: MCP CAPABILITIES

What a server can offer — and how clients find it.



SO FAR

- ✓ MCP standardizes AI-to-capability access
- ✓ Host, client and server have distinct roles
- ✓ Messages are JSON-RPC

NEXT QUESTION

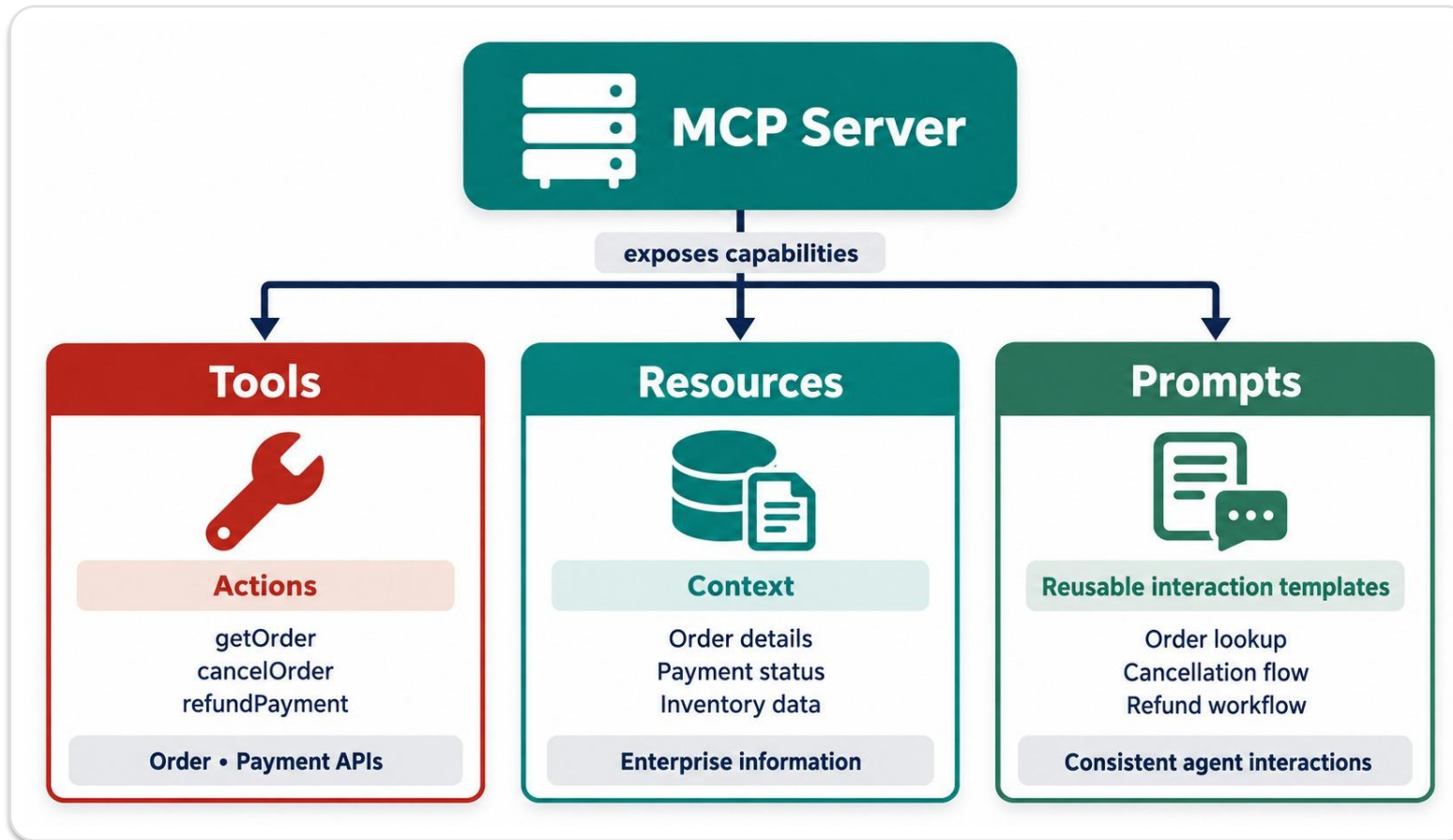
What can an MCP server expose — and how do clients learn about it?

IN THIS PART

- 1 Tools, resources and prompts
- 2 Tools vs. resources for microservices
- 3 Enterprise context through resources
- 4 Discovery and capability listing
- 5 Exercise: tool, resource or prompt?

MCP CAPABILITIES: TOOLS, RESOURCES, AND PROMPTS

Different ways servers expose functionality and context.



Tools

Model-invoked actions: get_order, request_refund.

Resources

Read-only context by URI: policies, runbooks, schemas.

Prompts

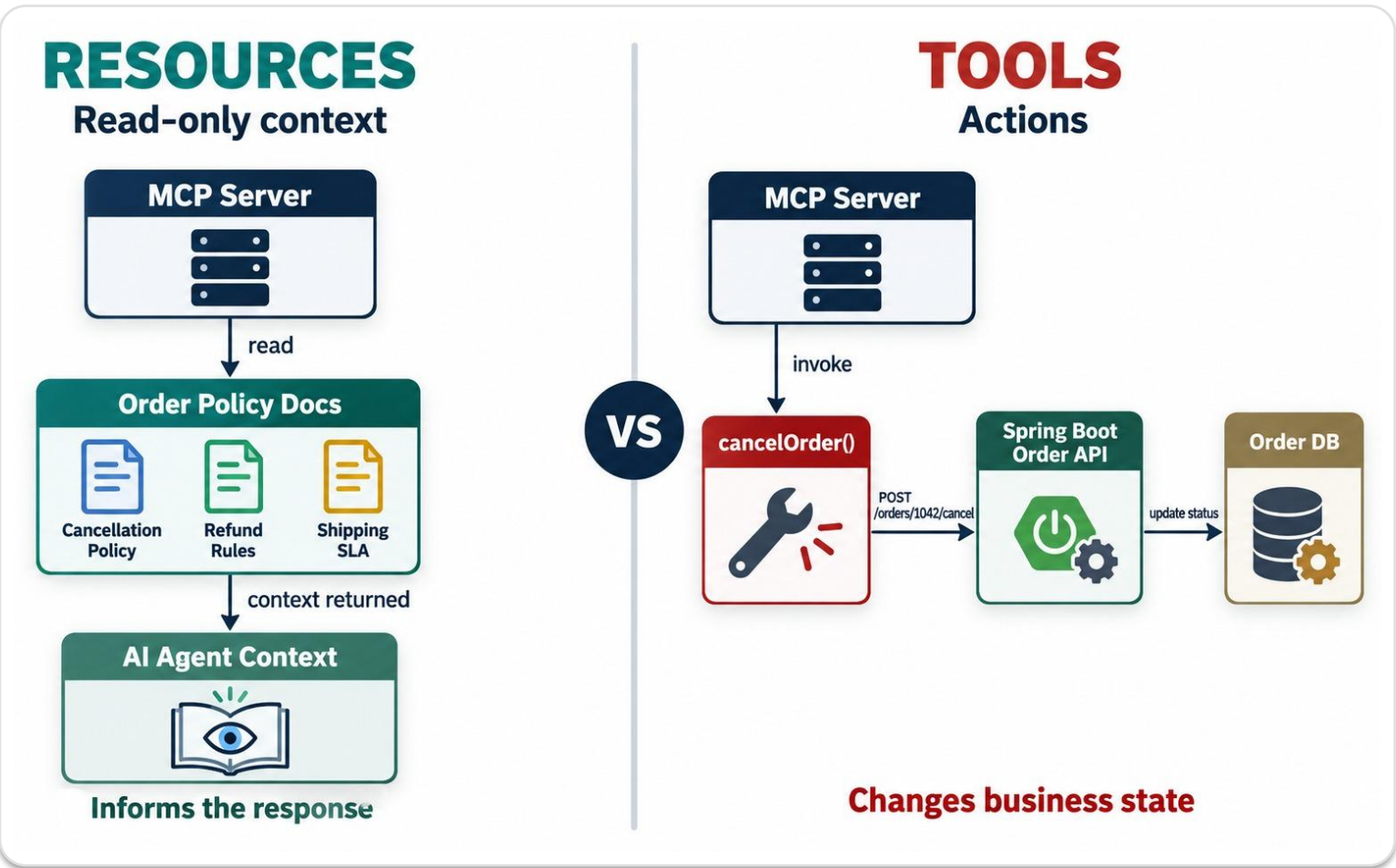
User-chosen templates: "Investigate a failed payment."

MENTAL MODEL Tools do, resources inform, prompts structure.

KEY TAKEAWAY Tools do, resources inform, prompts structure — servers can expose all three.

TOOLS VS. RESOURCES IN MICROSERVICES

Actions versus read-oriented context.



Resource

policy://refunds — what the refund rules say.

Tool

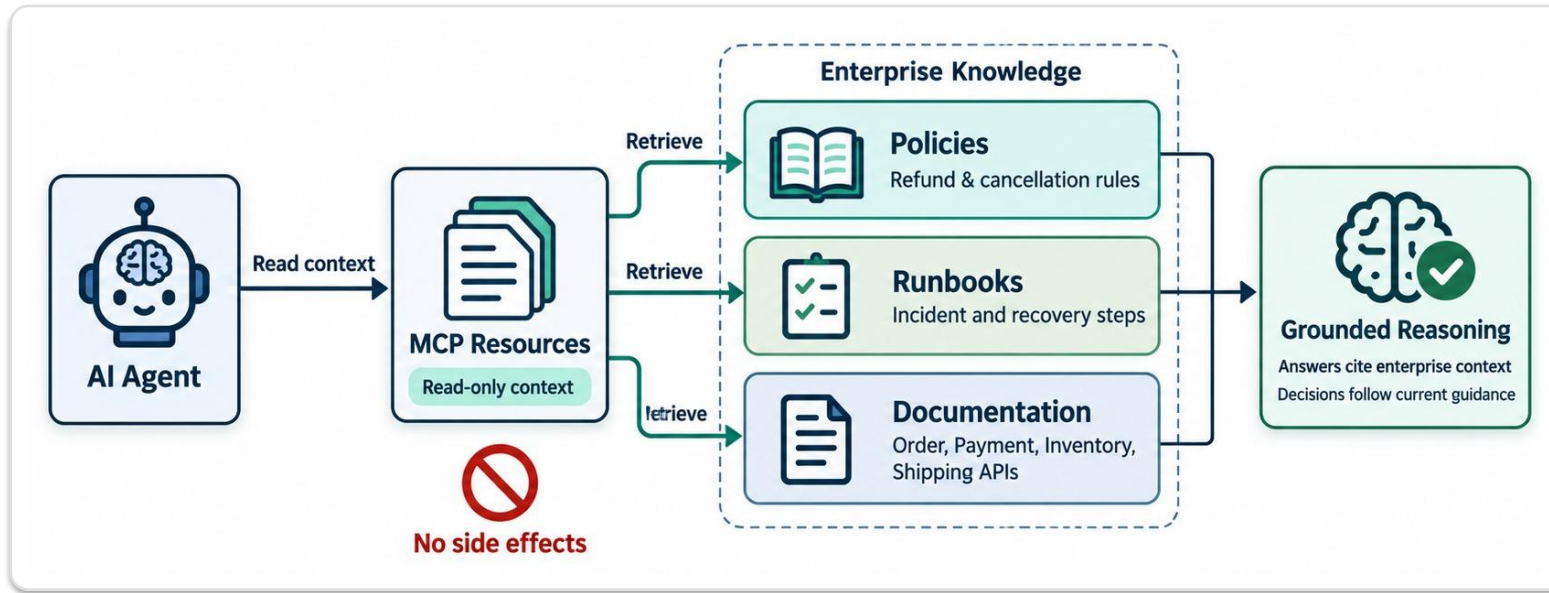
request_refund — actually starts a refund, under authorization.

RULE Knowing a policy never grants the right to act on it.

KEY TAKEAWAY Resources provide context; tools provide capability.

MCP RESOURCES FOR ENTERPRISE CONTEXT

Exposing read-only business and reference information safely.



A resource entry

```
{ "uri": "policy://payments/refunds/v3",  
  "name": "Refund policy",  
  "mimeType": "text/markdown" }
```

GOOD RESOURCES

Policies

Runbooks

API docs

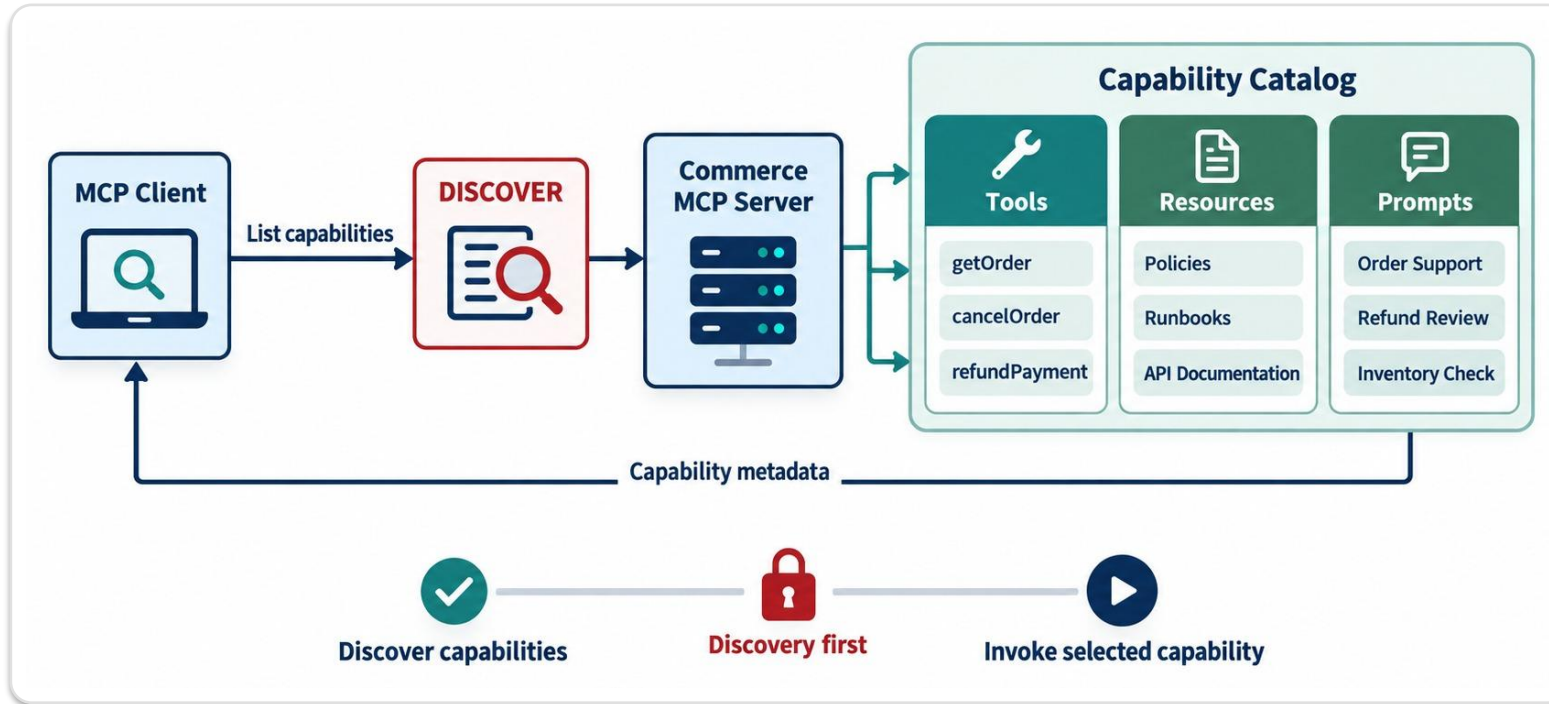
Error catalogs

NOT FOR LIVE STATE Order status comes from a tool.

KEY TAKEAWAY Resources give reference context; live business state still comes from services.

MCP DISCOVERY AND CAPABILITY LISTING

How clients discover available server capabilities.



Initialize

Client and server agree on protocol version and features.

List

tools/list, resources/list and prompts/list return the catalog.

Change

Servers can notify clients when the list changes.

CACHE Reuse the catalog while it's unchanged.

KEY TAKEAWAY Clients discover what a server offers — and cache it while it's unchanged.

EXERCISE: TOOL, RESOURCE OR PROMPT?

Classify each capability for an order-support MCP server.

PRACTICE EXERCISE

8 MIN

SCENARIO

The team wants to expose:

- 1 Current status of an order
- 2 The refund policy document
- 3 “Investigate a failed payment” steps
- 4 Start a refund
- 5 The error-code catalog
- 6 Reserve stock for an order

YOUR TASKS

- 1 Label each: tool, resource or prompt
- 2 Mark which tools change state
- 3 Say which need human approval

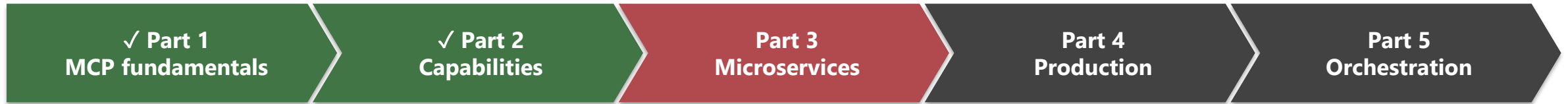
EXPECTED OUTCOME

- ✓ Tools: 1, 4, 6
- ✓ Resources: 2, 5
- ✓ Prompt: 3
- ✓ State-changing: 4 and 6
- ✓ Approval: 4 always; 6 by policy

KEY TAKEAWAY Live data and actions are tools, reference content is a resource, reusable workflows are prompts.

PART 3: MCP AND MICROSERVICES

Put an MCP layer in front of the services we already built.



SO FAR

- ✓ Tools act; resources inform; prompts structure
- ✓ Clients discover capabilities at runtime
- ✓ Actions are separated from context

NEXT QUESTION

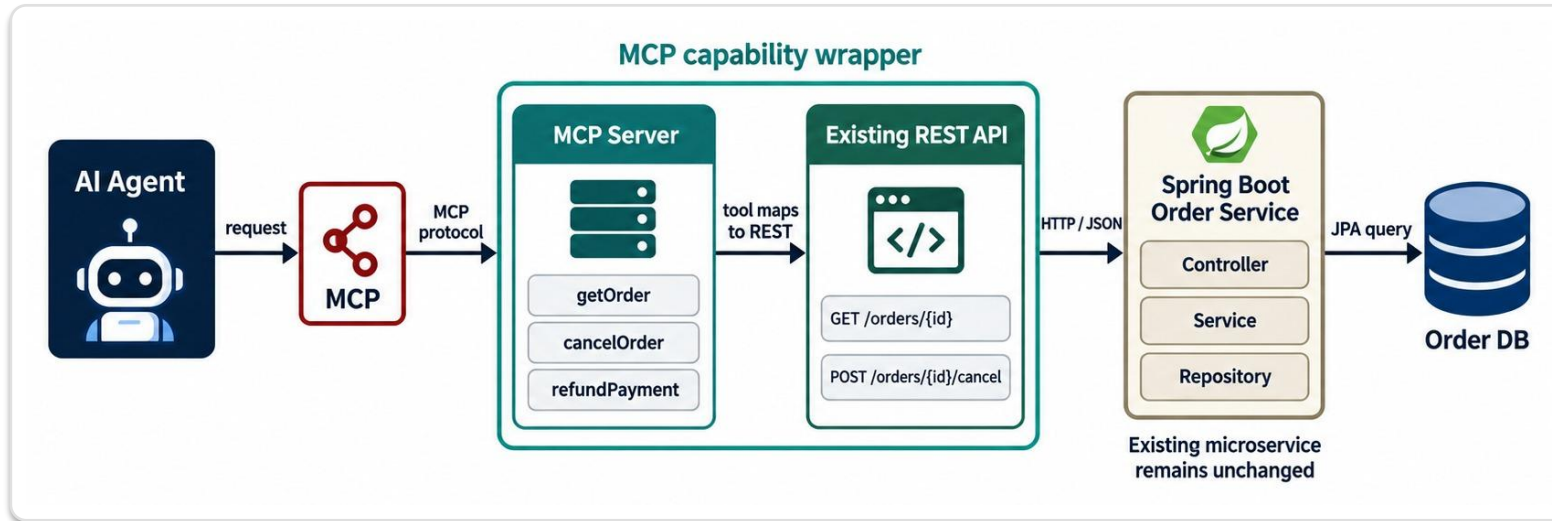
How do we expose our Spring Boot microservices to agents through MCP?

IN THIS PART

- 1 MCP and REST together
- 2 Calling Spring Boot through MCP — with Spring AI
- 3 Server design across services
- 4 Mapping APIs to tools
- 5 Tool contracts — with a schema
- 6 Exercises: MCP or REST? Map APIs to tools

MCP AND TRADITIONAL REST APIS

MCP isn't a replacement for REST — it can wrap or orchestrate APIs.



REST keeps serving

Web apps, mobile apps and other services call the APIs directly.

MCP adds

An AI-friendly layer: descriptions, schemas, discovery.

MESSAGE MCP complements REST.

KEY TAKEAWAY MCP complements REST. It does not automatically replace it.

EXERCISE: MCP OR REST?

Decide which clients should use MCP — and which should stay on REST.

PRACTICE EXERCISE

8 MIN

SCENARIO

Consumers of the order platform:

- A Mobile app showing order history
- B Support copilot answering questions
- C Payment Service calling Order
- D IDE agent helping ops engineers
- E Nightly reconciliation batch

YOUR TASKS

- 1 Choose MCP or REST for each
- 2 Explain the reason in a few words
- 3 Say what stays the same either way

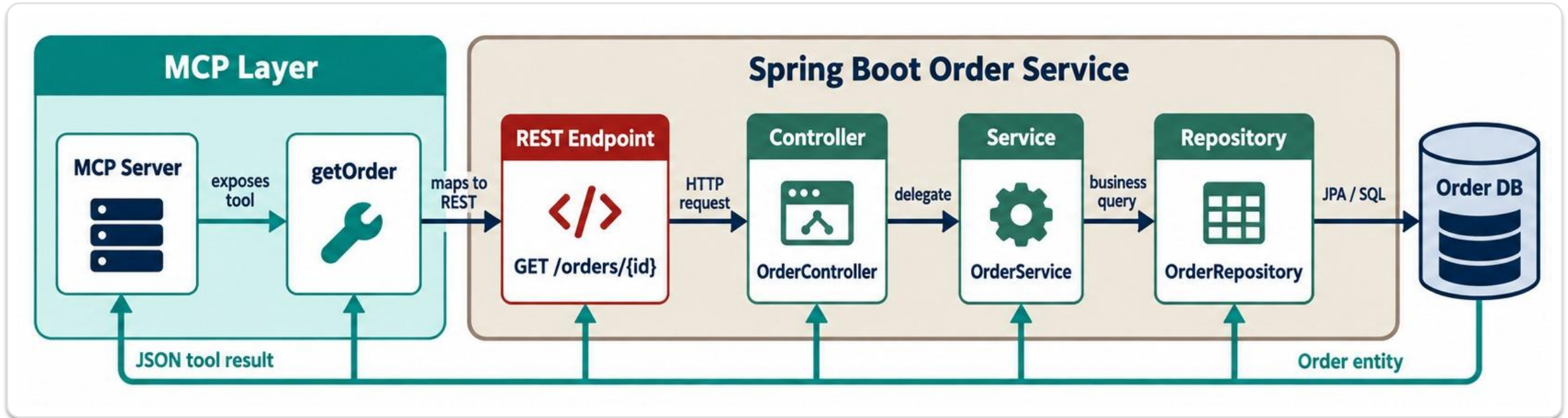
EXPECTED OUTCOME

- ✓ A, C, E: REST — fixed, known calls
- ✓ B, D: MCP — AI clients discovering tools
- ✓ Both reach the same Order API
- ✓ Same authorization and validation
- ✓ No database shortcuts for either

KEY TAKEAWAY Use MCP for AI clients that discover capabilities; keep REST for fixed, programmatic integrations.

CALLING A SPRING BOOT MICROSERVICE THROUGH MCP

Agent → MCP tool → REST endpoint → Spring Boot service.



What the agent sees

`get_order(orderId)` — not URLs, headers or JSON mapping.

The server's job

Map the tool to the API, pass identity, shape the result.

ARCHITECTURE RULE Call the service API — never its database.

KEY TAKEAWAY MCP should call the service API — not bypass the service to reach its database.

AN MCP SERVER WITH SPRING AI

Annotated methods become MCP tools that call the existing Order API.

✓ OrderTools.java

```
@Service
class OrderTools {
    private final RestClient orderApi;    // http://order-service:8080

    OrderTools(RestClient.Builder builder) {
        this.orderApi = builder.baseUrl("http://order-service:8080").build();
    }

    @Tool(name = "get_order",
          description = "Status and items of one order. Read-only.")
    OrderSummary getOrder(
        @ToolParam(description = "Order id, e.g. ORD-1045") String orderId) {
        return orderApi.get()
            .uri("/api/v1/orders/{id}", orderId)
            .retrieve()
            .body(OrderSummary.class);
    }
}
```

Register and configure

```
@Bean
ToolCallbackProvider orderToolProvider(OrderTools tools) {
    return MethodToolCallbackProvider.builder()
        .toolObjects(tools)
        .build();
}

# pom.xml: spring-ai-starter-mcp-server-webmvc
# application.yml
spring:
  ai:
    mcp:
      server:
        name: order-mcp-server
        version: 1.0.0
```

@Tool

Name and description become the MCP tool definition.

RestClient

The tool calls Order Service's API, with its rules.

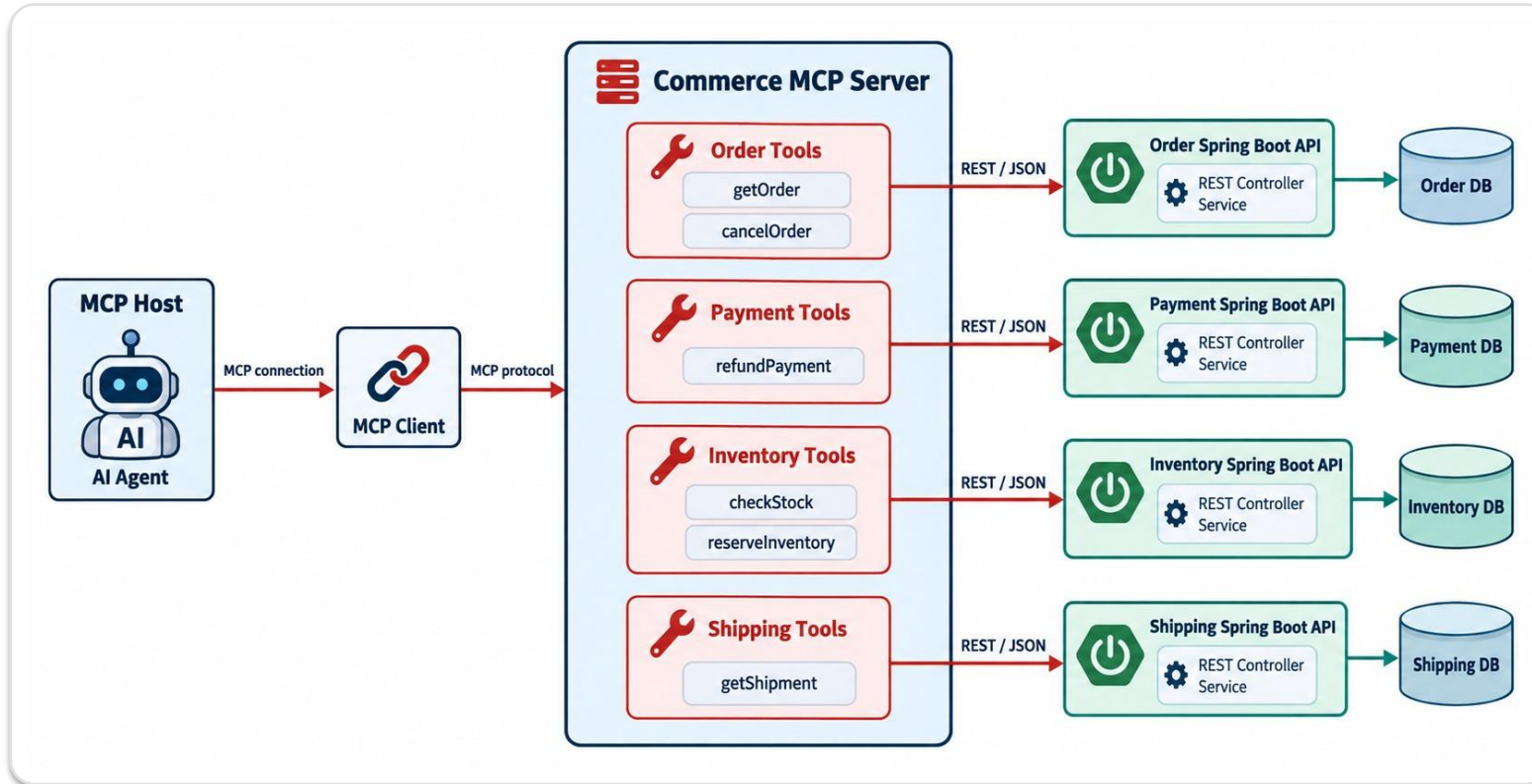
Starter

The Spring AI MCP server starter handles the protocol.

KEY TAKEAWAY A Spring AI MCP server is a thin adapter: annotated tools calling your existing service APIs.

DESIGNING AN MCP SERVER FOR MICROSERVICES

An MCP adapter layer in front of Order, Payment and Inventory services.



One server per domain

Owned by each service team; clear ownership and permissions.

One aggregated server

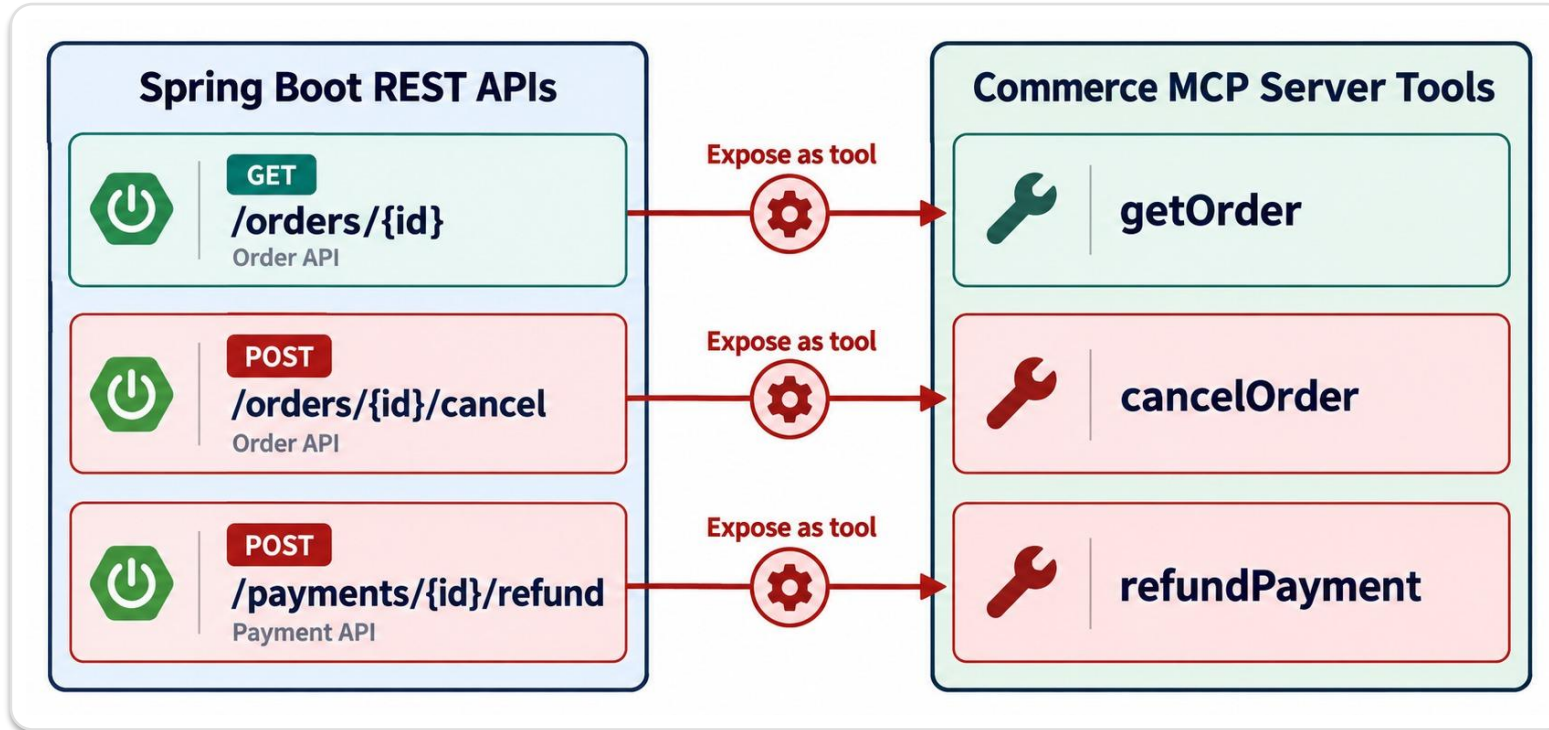
Simpler for clients; watch it doesn't become a monolith.

MODULE 2 Choose server boundaries like service boundaries.

KEY TAKEAWAY Choose MCP server boundaries deliberately — weigh convenience against ownership.

MAPPING MICROSERVICE APIS TO MCP TOOLS

getOrder, cancelOrder, refundPayment and other meaningful capabilities.



MAP CAPABILITIES

- ✓ GET /orders/{id} → get_order
- ✓ POST /orders/{id}/cancel → cancel_order
- ✓ POST /payments/{id}/refunds → request_refund

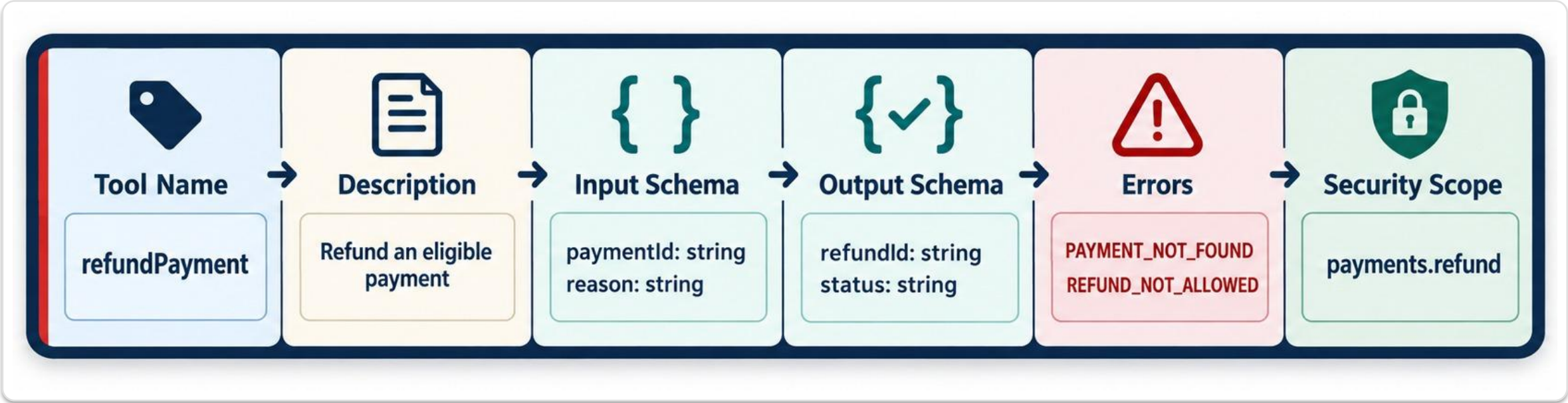
KEEP OUT

- ✗ Internal admin endpoints
- ✗ Bulk delete
- ✗ Health and metrics

KEY TAKEAWAY Expose meaningful business capabilities as tools — not every endpoint.

DESIGNING GOOD MCP TOOL CONTRACTS

Names, descriptions, schemas, inputs, outputs and errors.



Name

verb_noun, specific: request_refund, not do_payment.

Schemas

Typed input and output with patterns and limits.

Errors

Safe, structured errors — never invented data.

TREAT IT AS AN API Version it and review changes.

KEY TAKEAWAY Design tool contracts as carefully as any enterprise API contract.

A TOOL CONTRACT FOR A STATE-CHANGING ACTION

Precise description, strict input, structured output and honest hints.

✓ request_refund — tool definition

```
{
  "name": "request_refund",
  "title": "Request a refund",
  "description": "Create a refund REQUEST for a captured payment.
    A person must approve it. Use only after get_payment shows
    status CAPTURED. Never for partial amounts.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "paymentId": { "type": "string", "pattern": "^PAY-[0-9]+$" },
      "reason": { "type": "string", "maxLength": 200 }
    },
    "required": ["paymentId", "reason"],
    "additionalProperties": false
  },
  "outputSchema": {
    "type": "object",
    "properties": {
      "refundRequestId": { "type": "string" },
      "status": { "enum": ["PENDING_APPROVAL", "REJECTED"] }
    }
  },
  "annotations": { "readOnlyHint": false, "destructiveHint": false,
    "idempotentHint": true }
}
```

Description

Says when to use it — and when not to.

Input

Pattern, length limit and no extra properties.

Output

A pending request, not a finished refund.

HINTS are advisory — the server still enforces rules.

KEY TAKEAWAY A good tool contract tells the model when to use it and lets the server reject anything else.

EXERCISE: MAP APIS TO TOOLS

Design the tool list for a commerce MCP server.

PRACTICE EXERCISE

10 MIN

SCENARIO

Existing endpoints:

```
GET  /api/v1/orders/{id}
POST /api/v1/orders/{id}/cancel
GET  /api/v1/payments/{id}
POST /api/v1/payments/{id}/refunds
GET  /api/v1/inventory/{sku}
DELETE /api/v1/admin/orders
GET  /actuator/metrics
```

YOUR TASKS

- 1 Choose which endpoints become tools — and name them
- 2 Mark each tool read or state-changing
- 3 List the endpoints you'd keep out, and why

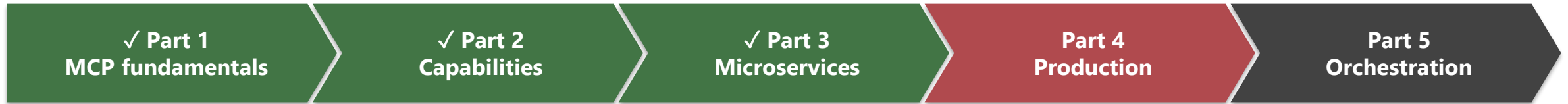
EXPECTED OUTCOME

- ✓ get_order, get_payment, check_stock: read
- ✓ cancel_order: state-changing, scoped
- ✓ request_refund: state-changing, approval
- ✓ Out: admin bulk delete
- ✓ Out: actuator metrics

KEY TAKEAWAY Expose a small set of well-named business tools; keep admin and operational endpoints out.

PART 4: PRODUCTION ARCHITECTURE

Run MCP servers like any enterprise workload — scaled, secured and observed.



SO FAR

- ✓ MCP servers wrap existing APIs
- ✓ Tools map to business capabilities
- ✓ Contracts are precise and enforced

NEXT QUESTION

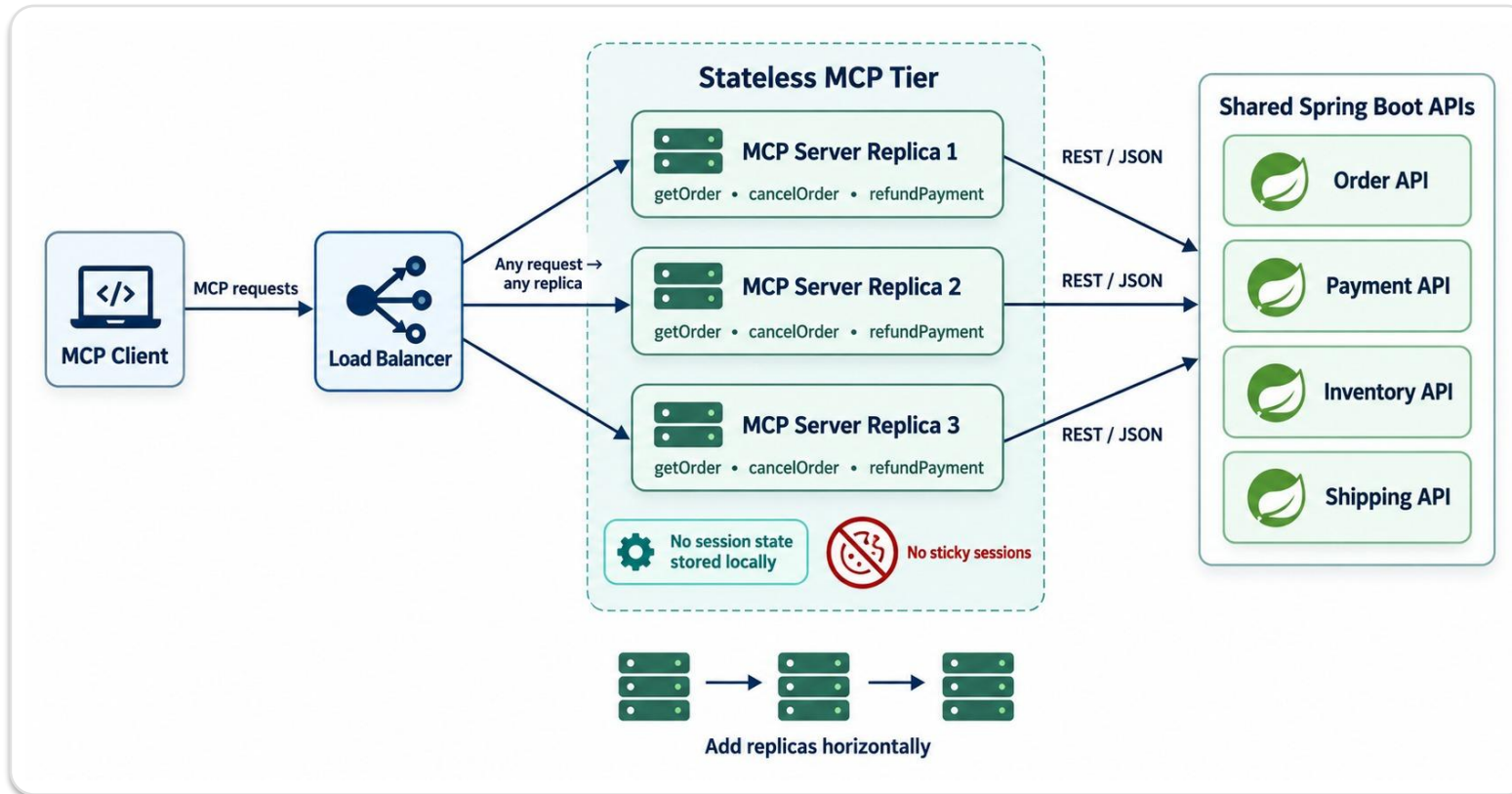
How do we run MCP servers safely in production?

IN THIS PART

- 1 Stateless servers and horizontal scaling
- 2 MCP behind an API gateway
- 3 Authentication and authorization — in code
- 4 Defense in depth
- 5 Exercise 5.3: MCP security controls
- 6 Reliability and observability

STATELESS MCP AND HORIZONTAL SCALABILITY

The current stateless core and load-balanced server instances.



Three replicas on OpenShift

```
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: order-mcp-server
          readinessProbe:
            httpGet: { path:
/actuator/health/readiness,
port: 8080 }
```

No sticky sessions

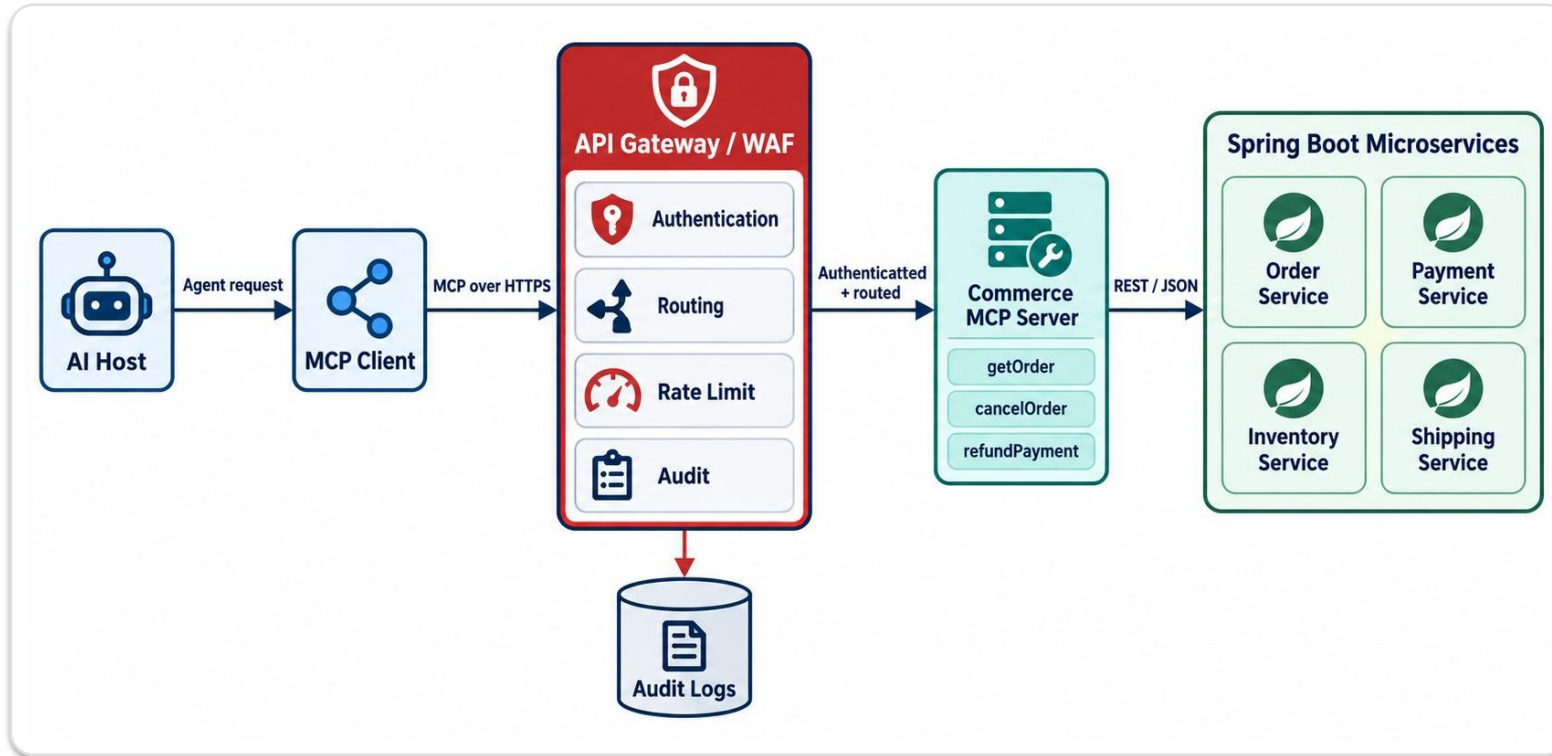
Any replica can serve any request.

MODULE 8 Same Deployments, probes and scaling.

KEY TAKEAWAY A stateless protocol core lets MCP servers scale like any cloud-native workload.

MCP BEHIND AN API GATEWAY

Routing, policy enforcement, rate limiting and tracing.



GATEWAY ENFORCES

TLS

Token validation

Rate limits

WAF rules

Routing

Audit logs

Agent traffic is bursty

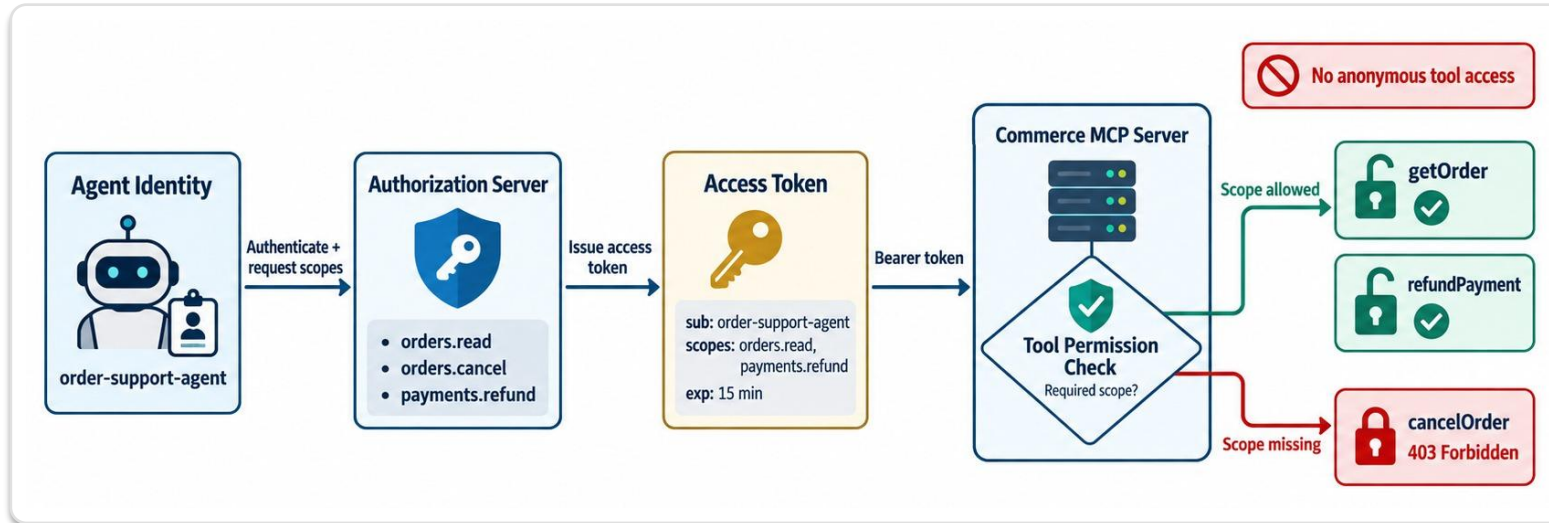
Rate-limit per agent and per user.

STILL the MCP server and services authorize too.

KEY TAKEAWAY Treat MCP traffic like enterprise HTTP traffic — gateways route and govern it.

AUTHENTICATION AND AUTHORIZATION FOR MCP

OAuth-based access, identity propagation and least privilege.



Authenticate

The MCP server is an OAuth resource server; no anonymous calls.

Authorize per tool

get_order needs orders.read; request_refund needs refunds.request.

Propagate identity

Pass the user's token or a scoped one — never a super-user.

MODULE 12 OAuth2, OIDC and JWT in depth.

KEY TAKEAWAY Authenticate the caller, authorize each tool — never hand the model super-user access.

SECURING TOOLS IN THE MCP SERVER

Validate the token, check a scope per tool, forward identity and audit the call.

SecurityConfig.java

```
@Bean
SecurityFilterChain mcp(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}
```

Resource server

Every MCP request carries a validated JWT.

✓ PaymentTools.java

```
@Tool(name = "request_refund", description = "...")
@PreAuthorize("hasAuthority('SCOPE_refunds.request')")
RefundRequest requestRefund(String paymentId, String reason) {
    Jwt caller = currentJwt();
    RefundRequest result = paymentApi.post()
        .uri("/api/v1/payments/{id}/refund-requests", paymentId)
        .headers(h -> h.setBearerAuth(caller.getTokenValue()))
        .header("Idempotency-Key", idempotencyKey(paymentId, reason))
        .body(new RefundReason(reason))
        .retrieve()
        .body(RefundRequest.class);
    audit.toolCall("request_refund", paymentId, caller.getSubject(),
        correlationId(), result.status());
    return result;
}
```

Per-tool scope

The model can't use a tool the caller lacks.

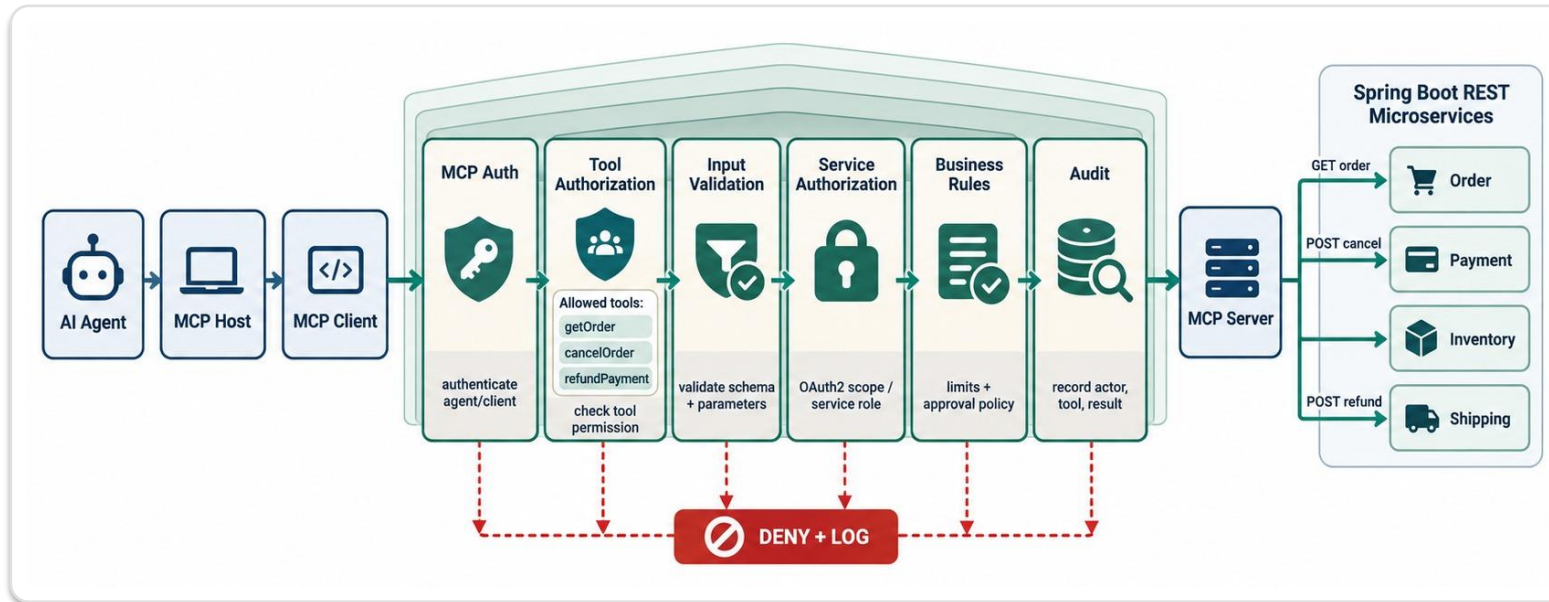
Forward identity

Payment Service authorizes the same user again.

KEY TAKEAWAY Secure MCP like any API: validate tokens, scope each tool, forward identity and audit every call.

SECURING MCP-ENABLED MICROSERVICES

Tool allow-lists, validation, service credentials and audit.



MCP-SPECIFIC THREATS

- ! Prompt injection in tool results
- ! Poisoned tool descriptions
- ! Over-powerful tools
- ! Data leaking into prompts

Defenses

Trusted servers only, strict schemas, output sanitizing.

CRITICAL RULE The microservice still authorizes everything.

KEY TAKEAWAY Defense in depth still applies — every layer checks, including the microservice.

EXERCISE 5.3 — MCP SECURITY CONTROLS FOR BANKING APIS

Day 5 checkpoint C: authentication, human-in-the-loop and audit for tool exposure.

OFFICIAL CHECKPOINT C

15 MIN

labs/day-05/lab5/starter/mcp-controls.md

SCENARIO

An assistant would call Lab 5's Risk Assessment Service through two MCP tools: `get_assessment` and `submit_review`.

Awareness only — no MCP runtime in the lab.

YOUR TASKS

- 1 Walk `mcp-controls.md`: who may call `get_assessment` and `submit_review`?
- 2 Which tool needs a human in the loop before money-movement effects?
- 3 Which audit fields would you persist (actor, tool, correlation ID, decision)?
- 4 Don't start an MCP process, Keycloak or a Microsoft agent runtime

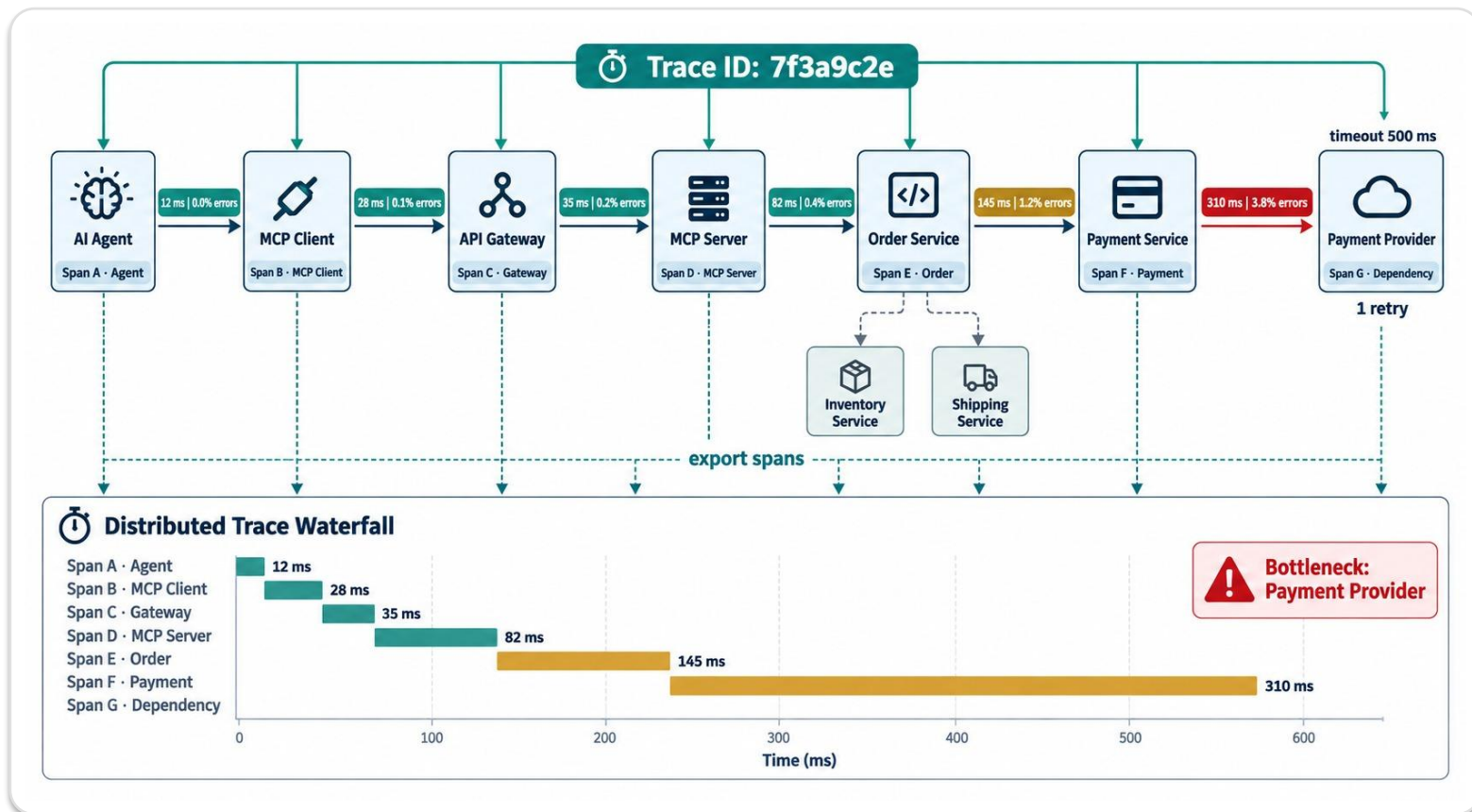
EXPECTED OUTCOME

- ✓ `get_assessment`: `risk.read`
- ✓ `submit_review`: `risk.write`, human-confirmed
- ✓ No superuser token; no anonymous calls
- ✓ Audit: tool, IDs, caller, correlation, outcome
- ✓ Never log tokens, keys or PAN

KEY TAKEAWAY Tools carry the user's identity, need narrow scopes, keep humans on reviews and leave an audit trail.

RELIABILITY AND OBSERVABILITY FOR MCP

Timeouts, retries, tracing, metrics and tool-call auditing.



Timeouts

Every backend call from a tool has one — Module 6.

Safe errors

Return `isError` with a clear reason; never guess the data.

TRACK

Tool latency

Tool error rate

Calls per agent

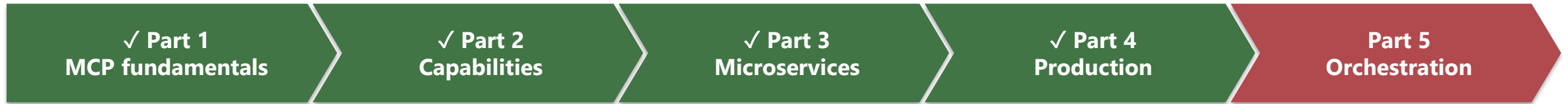
Trace ID

KEY QUESTION Which agent run caused this service call?

KEY TAKEAWAY An MCP tool call is a distributed request — make every hop resilient and traceable.

PART 5: AGENTIC ORCHESTRATION

One agent, one protocol, many services — each keeping its authority.



SO FAR

- ✓ MCP servers scale statelessly
- ✓ Tokens, scopes and audit protect tools
- ✓ Every tool call is traced

NEXT QUESTION

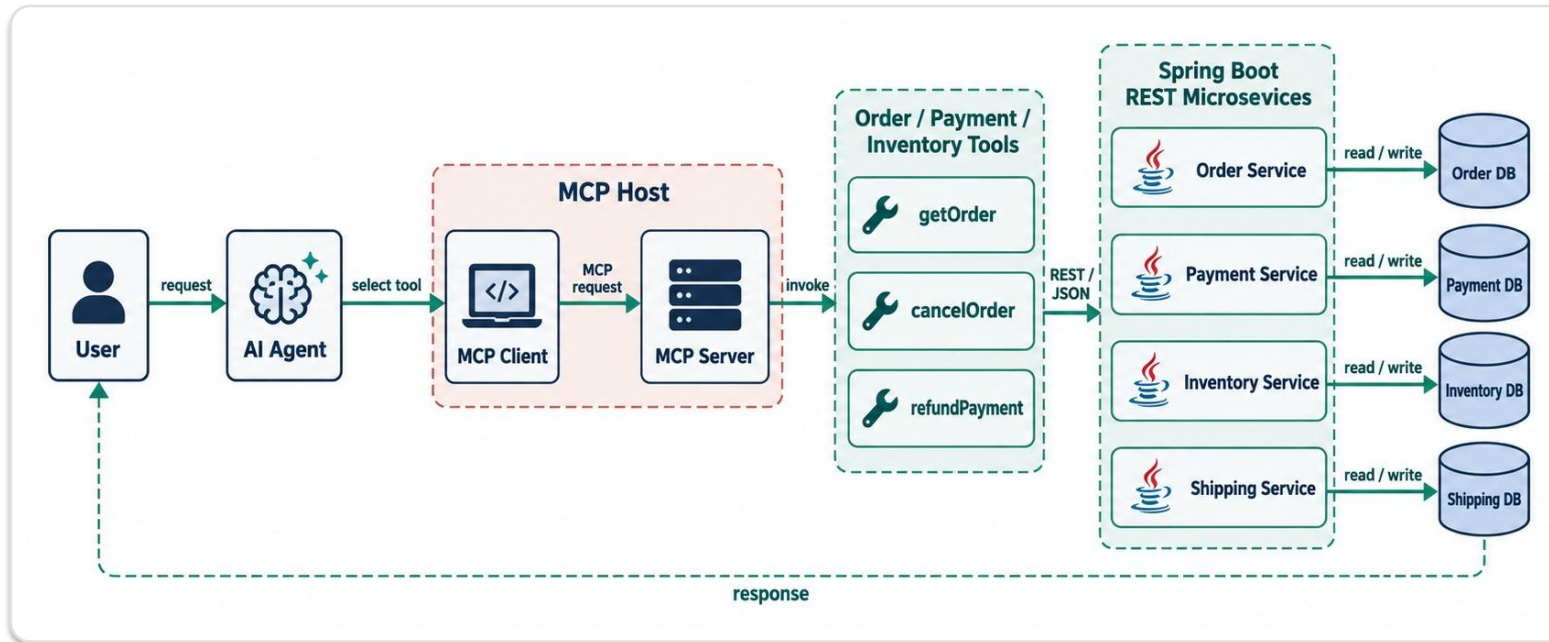
How does an agent complete a business task across several services through MCP?

IN THIS PART

- 1 Agent + MCP + Spring Boot architecture
- 2 Multi-service orchestration
- 3 Long-running operations and tasks
- 4 Exercise: review a risky tool
- 5 Use cases and the complete architecture

AI AGENT + MCP + SPRING BOOT ARCHITECTURE

An agent using MCP to safely interact with multiple microservices.



Agent

Understands intent and plans the steps.

MCP layer

Exposes tools, checks scopes, forwards identity.

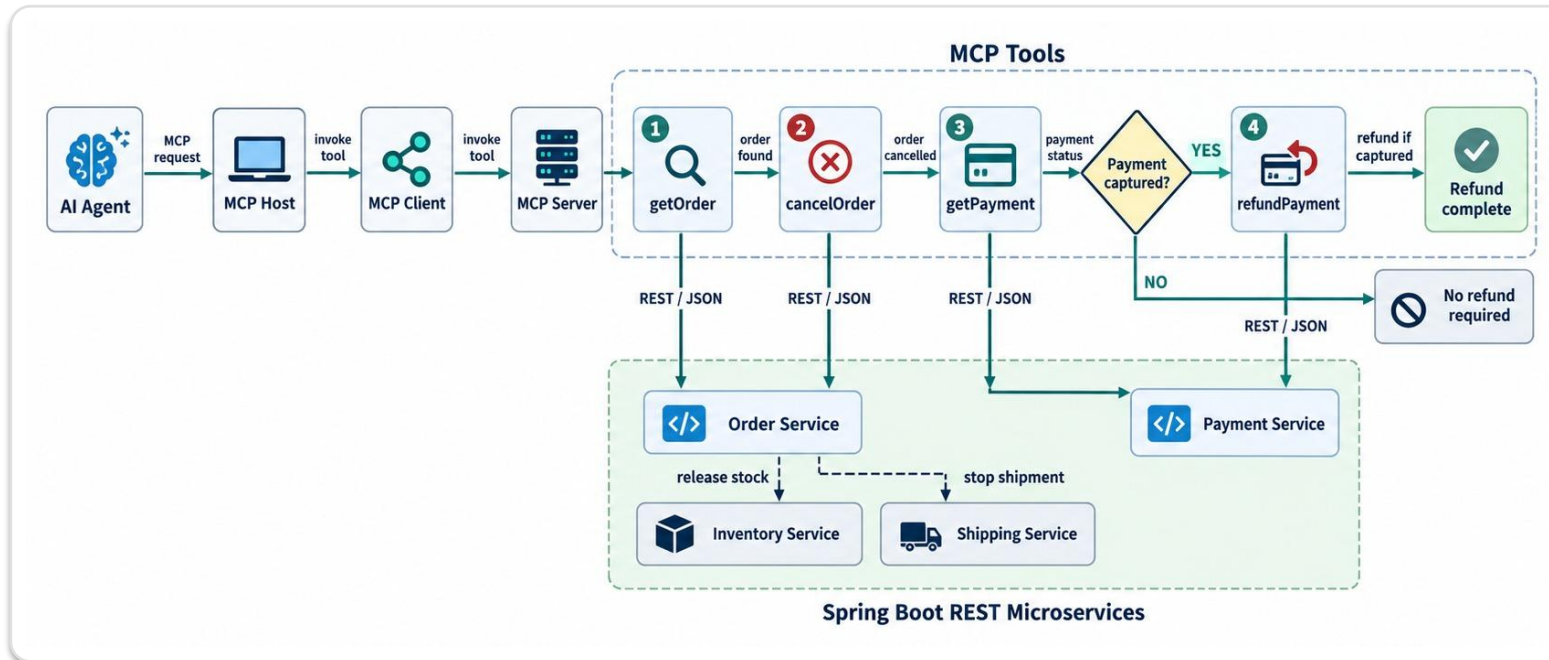
Microservices

Validate, enforce rules, own the data.

KEY TAKEAWAY Agents use a standard MCP capability layer instead of custom integration code per backend.

MCP FOR MULTI-SERVICE API ORCHESTRATION

One agent task invoking Order, Payment and Shipping tools.



THE DECISION PATH

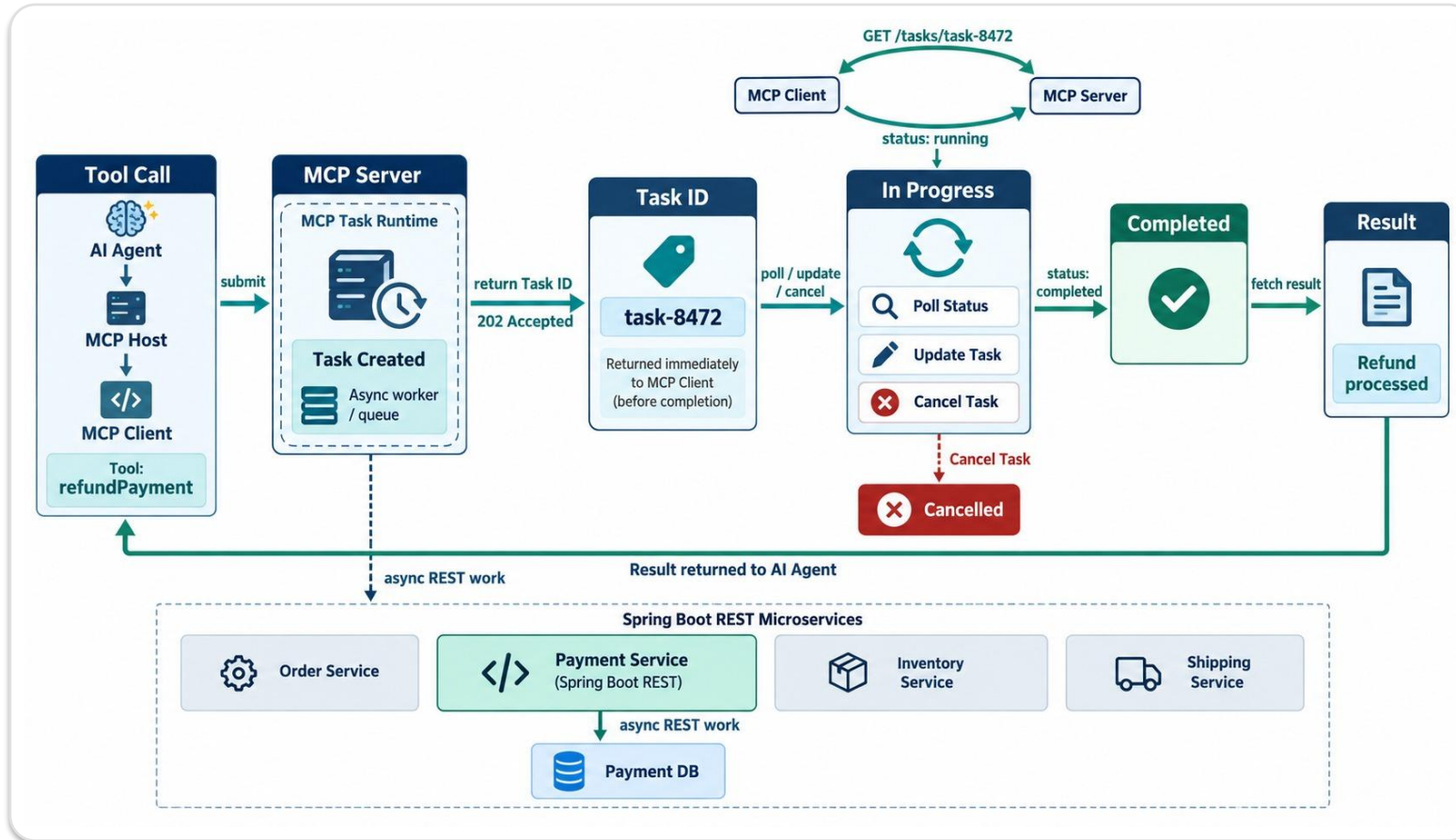
- ✓ **get_order**: is it cancellable?
- ✓ **cancel_order**
- ✓ **get_payment**: was it captured?
- ✓ **request_refund** — only if captured

SUMMARY The agent orchestrates, MCP connects, services decide.

KEY TAKEAWAY The agent orchestrates, MCP connects, and each microservice keeps business authority.

LONG-RUNNING OPERATIONS AND MCP TASKS

Handling operations that don't complete immediately.



Examples

Bulk reconciliation, report generation, a multi-step refund.

Task handle

Return an ID at once; the client polls or is notified.

Cancel

Clients can cancel a task that's no longer needed.

SECURITY Only the caller who started it may read or cancel.

KEY TAKEAWAY Tasks turn long tool calls into trackable, cancellable asynchronous work.

EXERCISE: REVIEW A RISKY TOOL

Find the problems before this tool reaches production.

PRACTICE EXERCISE

10 MIN

SCENARIO

A team proposes this MCP tool:

```
name: run_payment_operation
description: Does payment stuff
input: { operation: string,
         sql: string }
auth: shared service account
      (payments admin)
effect: executes immediately
```

YOUR TASKS

- 1 Find at least five problems
- 2 Explain the risk of each
- 3 Redesign it as safe tools

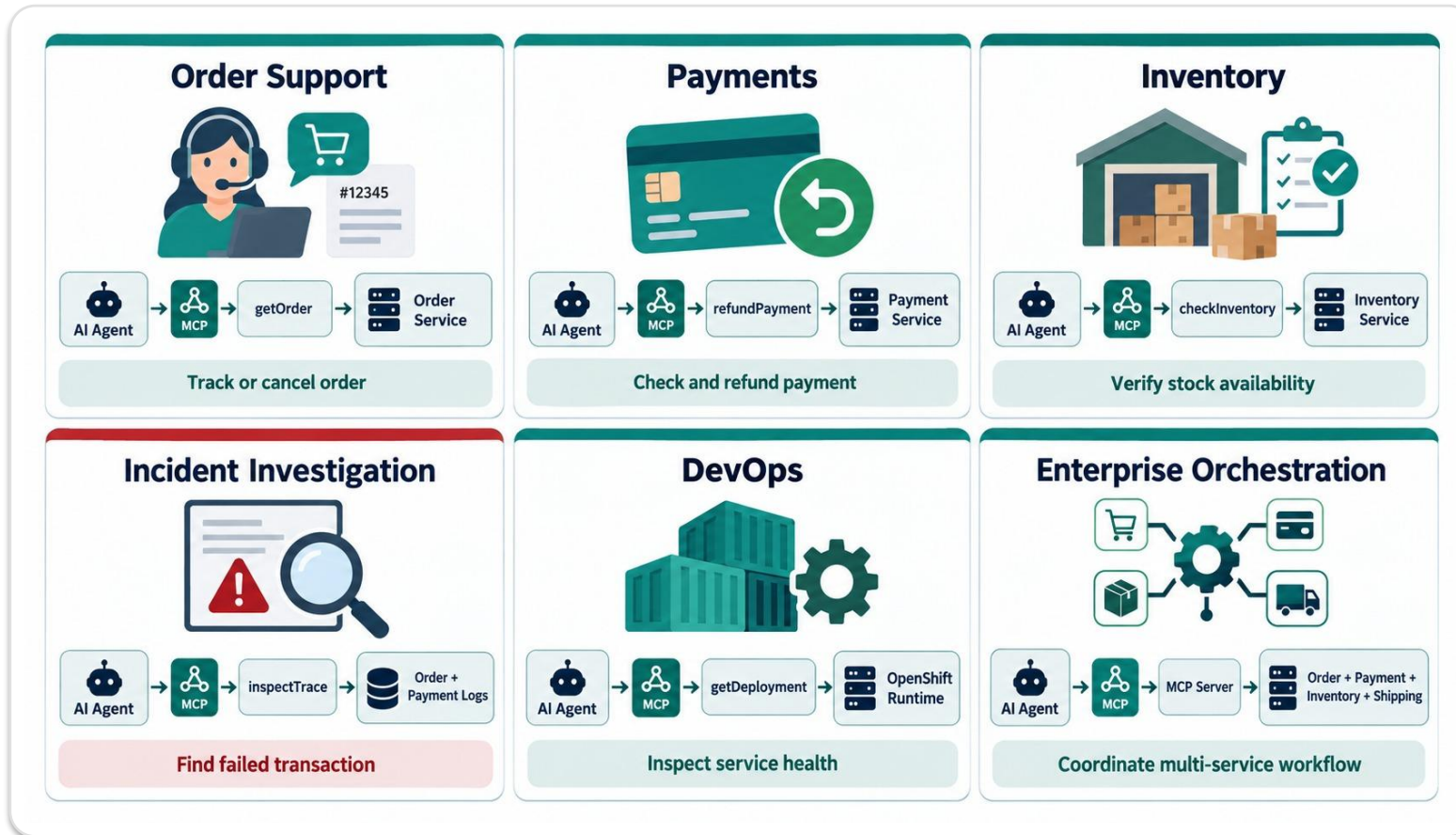
EXPECTED OUTCOME

- ✓ Vague name and description
- ✓ Free-form SQL → injection, bypasses the API
- ✓ Shared admin account → no least privilege
- ✓ Executes immediately → no approval
- ✓ Redesign: get_payment + request_refund

KEY TAKEAWAY Replace one all-powerful tool with narrow, typed, scoped tools — and approvals for risky ones.

PRACTICAL MCP PATTERNS AND USE CASES

Support agent, DevOps agent, order operations and incident analysis.



GOOD FITS

- ✓ Many AI clients need the same capabilities
- ✓ Capabilities are well governed
- ✓ Actions have clear owners

USE PLAIN REST

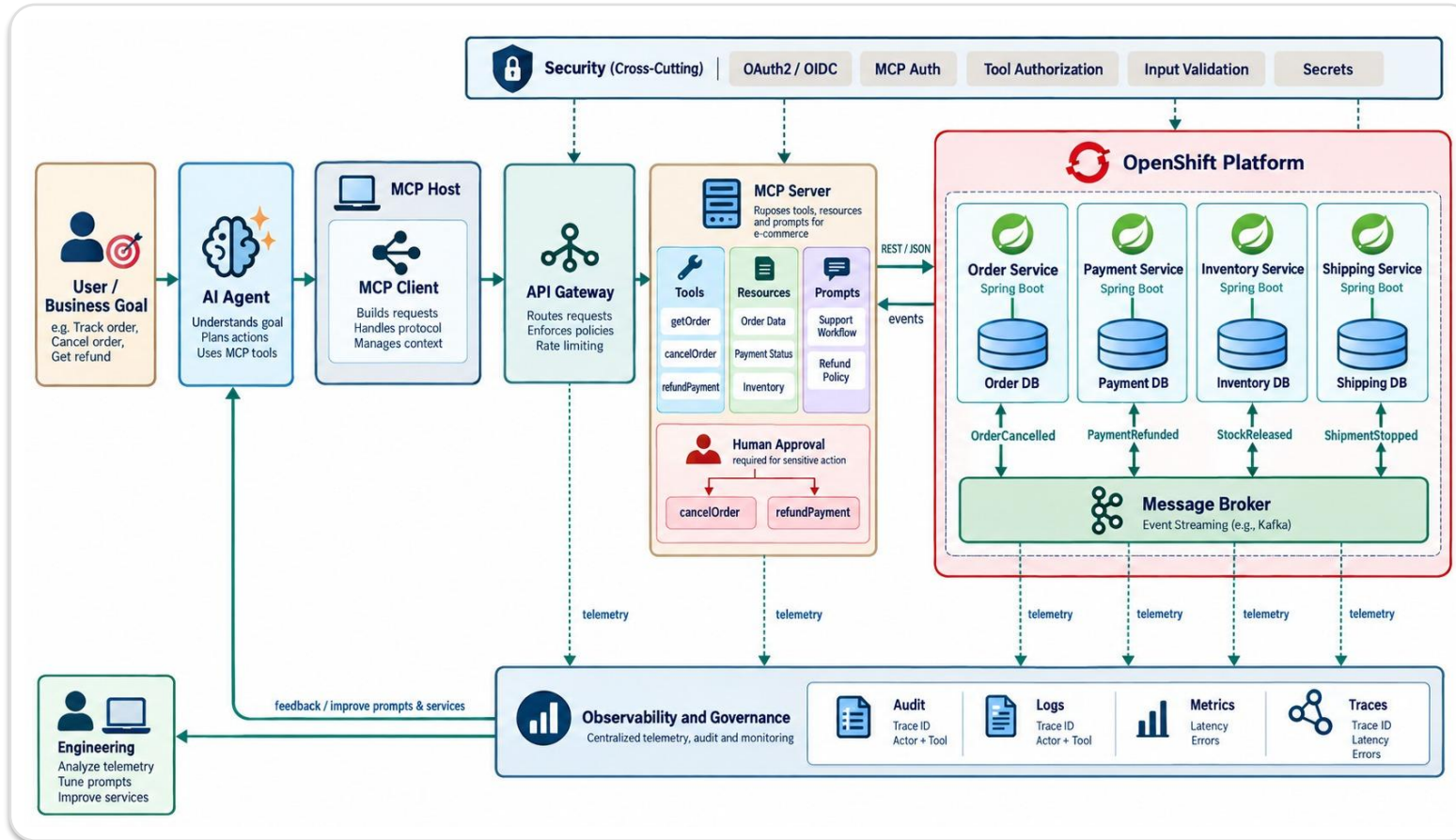
- ✗ Service-to-service calls
- ✗ One fixed integration

START with read-only tools, then add approved actions.

KEY TAKEAWAY MCP pays off most when many AI clients need consistent access to controlled capabilities.

END-TO-END MCP-ENABLED MICROSERVICES ARCHITECTURE

Agent → MCP → gateway → Spring Boot → data, events and OpenShift.



EVERY PART OF THE MODULE

- ✓ Agent discovers tools through MCP
- ✓ Gateway authenticates and limits
- ✓ Server authorizes and forwards identity
- ✓ Services enforce rules and own data
- ✓ Audit, traces and metrics throughout

REMEMBER The microservice remains the business authority.

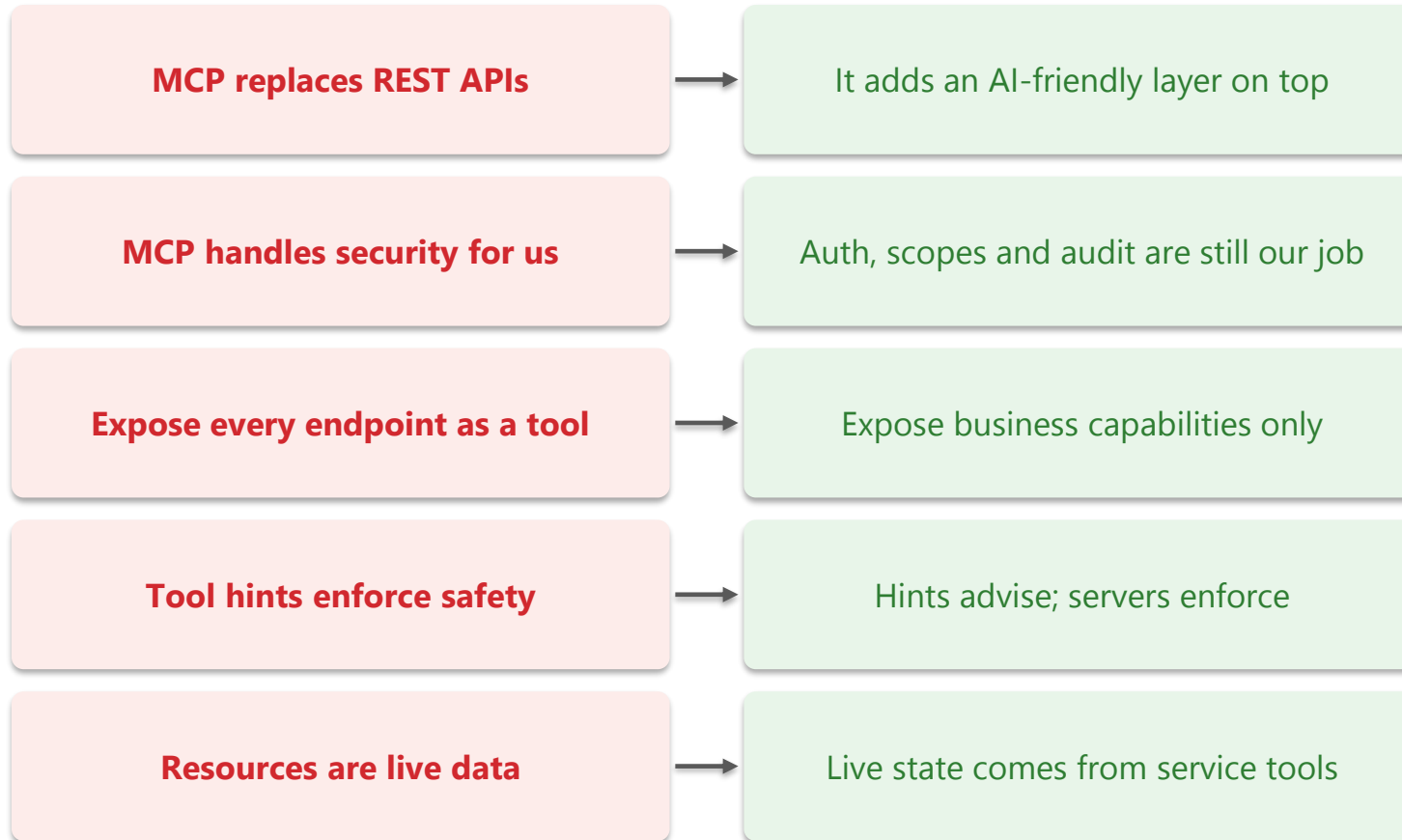
KEY TAKEAWAY The agent reasons, MCP connects, and the microservice remains the business authority.

MCP MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that make MCP integrations fragile or unsafe.

MISCONCEPTION

REALITY



ANTI-PATTERNS TO AVOID

- ✗ MCP server reading service databases
- ✗ One super-user token for all tools
- ✗ Vague tool names and descriptions
- ✗ Free-form SQL or command tools
- ✗ Untrusted third-party MCP servers
- ✗ No correlation between agent and service logs

REMEMBER Agent reasons, MCP connects, service decides.

KEY TAKEAWAY MCP is only as safe as its tool design, identity handling and the services behind it.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What is MCP used for?

2 What does an MCP client do?

3 What does an MCP server do?

4 What is an MCP tool?

5 Why keep tools narrow and task-specific?

6 Why call APIs instead of service databases?

7 Read tool vs. action tool?

8 Why do correlation IDs matter for MCP?

9 What if a backend service times out?

10 Why require approval for sensitive actions?

MODULE 11 SUMMARY AND WHAT'S NEXT

From custom connectors to a standard, secure capability layer for agents.

YOU CAN NOW

- ✓ Explain MCP hosts, clients, servers and messages
- ✓ Choose between tools, resources and prompts
- ✓ Wrap Spring Boot APIs as well-designed MCP tools
- ✓ Scale, secure and trace MCP servers
- ✓ Orchestrate several services through MCP safely

NEXT: MODULE 12 — SECURITY AND COMPLIANCE

- 1 How do OAuth2 and OIDC protect every hop?
- 2 How do we validate JWTs in Spring Boot?
- 3 How do we manage secrets and prove compliance?

BRING ALONG The scopes you designed for MCP tools — Module 12 enforces them.

KEY TAKEAWAY Module 12 secures every identity in this picture — users, services, agents and pipelines.